

Image Processing in Python - Introduction

This module assumes knowledge of the python programming language and familiarity with a python development environment. A principal aim is to allow you to develop your own solutions to a range of image processing problems, using available implementations of core techniques.

We will be using **OpenCV (version 4.1.x)** computer vision library with **Python 3.7.x**. Whilst OpenCV is primarily aimed at computer vision tasks (i.e. image understanding and analysis), it is built on a central image processing and manipulation component that is both well-documented, open-source and free. It is also the only library offering coverage of all of the course topics with interfaces in Python (used here) and Java. It has already been setup for use on the Lab PCs (Windows and Linux) but you may alternatively use your own development environment on your own hardware.

OpenCV has full documentation available at: <https://docs.opencv.org/4.1.1/>

Part 1: A First OpenCV Example with Python

Download the python example [smooth_image.py](#) from Learn Ultra and an example image of your choice from the internet (put them in the same directory). If you are struggling for inspiration on an example image use [peppers.png](#) (also on Learn Ultra), or another image from the SIPI database, which provides some of the most commonly-used standard test images used for image processing evaluation at: <http://sipi.usc.edu/database/database.php?volume=misc>

In the example code, change **line 20** of the example (if needed) to reference your image file and run the code. Spend some time understanding the program structure, objects and the flow of control. This program simply loads and displays in a window an image, specified by a string file name, after performing some mild smoothing on it.

Look up the [imread\(\)](#) method in the manual to find out how it operates and the image file formats it supports. Like all open-source software, the documentation is not perfect, but this is the real world!

Operations on Images

Next we examine how we use OpenCV functionality to perform specified operations on images. The key concepts to take note of are:

- **Automatic initialisation** of a new (output) image objects prior to use.
- Common way of **interfacing the provided OpenCV functions** to images and parameters.

As should be apparent from the source code ([smooth_image.py](#)) and running the program, this program applies a smoothing filter (here, Gaussian blurring) to the input image and displays the result. The details of this filter are provided here:

<http://homepages.inf.ed.ac.uk/rbf/HIPR2/gsmooth.htm>

We will cover this image filtering technique, later in the course. At the moment, we are just using it as an example of an operation performed on an image object.

Programming Tasks

1. Look up the entry for the `GaussianBlur()` function in the OpenCV manual.
 - Explore the effects of varying either of the mask (i.e. kernel) parameters from 5.
 - Try larger and smaller values (which must always be odd). What happens?
 - How does this affect the direction of the blurring?
2. Extend the program to use multiple window objects – to display the original input image as well as smoothed outputs with different parameters!
3. Investigate the use of the following OpenCV functions by replacing the `GaussianBlur()` operation in the code:

(a) `flip()`

(b) `Canny()` – an edge detection method. An example was briefly shown in Lecture 1.

You will need to first convert the image to grayscale using:

`cvtColor(inputImage, cv2.COLOR_BGR2GRAY)`

N.B. Suitable threshold parameters for the canny operator would be 10 and 200. You can leave the other parameters at the suggested defaults from the manual (or play around and experiment!!)

(c) `resize()`

N.B. You can just specify the `fx` and `fy` resize scales and let the function compute the required destination image size.

Investigate the effects of changing type of interpolation used (for example between `cv2.INTER_NEAREST` and `cv2.INTER_LINEAR`).

Use the OpenCV manual for reference. Experiment with different parameters and different images.

Part 2: Interacting with Image Data

The previous exercise illustrated the high-level use of an OpenCV function on an image object. However, to design and implement our own image processing functions, we have to be able to access the pixel data from the image directly. This example will explore accessing the raw image pixels directly through the OpenCV image object. In addition, it will illustrate some of the basic graphical user interface (GUI) functionality of the OpenCV library.

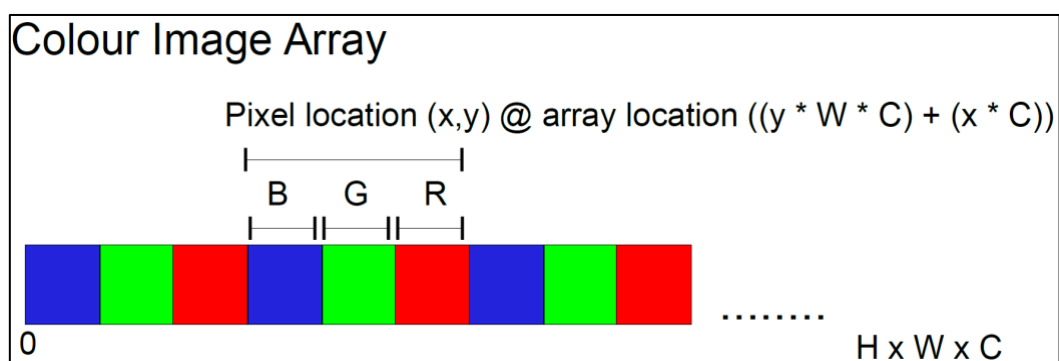
The key concepts to take note of are:

- Accessing pixel data through the **image matrix data object** within the pixel.
This hides the details of pixel access which we will discuss later!
- Using a **callback function to handle mouse events**.
This function is called for every mouse event in the assigned window.
- Use of the **waitkey()** function to capture keyboard input.
The window event loop can be used to capture keyboard input.

Download the python example [colour_query.py](#) from Learn Ultra and an example image of your choice from the internet (to the same directory) as before.

This example allows the user to click into the image window at a given (x,y) pixel location with the mouse and either retrieve or set the image colour for that pixel. Left clicking prints out the colour information to standard output and right clicking sets the colour of that pixel location to white.

Accessing the pixel element colour data is carried out using the image matrix data structure of the OpenCV python interface, which indexes a given pixel location, from the 2D image, into the actual 1D image storage array in memory. The image array is laid out as a consecutive array in memory of length $W \times H \times C$ for an image of width W , height H and with C colour channels. For colour images, the number of channels is $C=3$ and they are ordered in BGR – Blue, Green, Red).



For convenience, we can access a pixel element at location (x,y) using the following constructs. For a (1 channel) grayscale image location image(x,y) is:

```
grey = img[y,x]
```

In OpenCV, the x-axis corresponds to image columns (along top) and the y-axis to image rows (down the side) – assuming a top left image origin! (which OpenCV uses)

For a 3-channel colour image, location image(x,y) is (one for each channel, in order BGR!):

```
blue  = img[y,x,0]
green = img[y,x,1]
red   = img[y,x,2]
```

Most of the images we will be dealing with will be of data type 8-bit unsigned (unsigned char) as they have one byte allocated per colour channel (i.e. 8-bits) with possible value range 0-255. Specialist images (often containing medical or 3D information) may have floating point (float) or integer (int) channel spaces. As shown in the example, an image object can be queried to return the number of channels that it contains thus allowing code to be designed for both grayscale and colour images.

Programming Tasks

1. Change the above code so that a right mouse click changes the current pixel colour to:

- (a) Black
- (b) Red
- (c) Green
- (d) Blue
- (e) Light Grey
- (f) Dark Grey
- (g) Yellow
- (h) Purple

Record the numerical colour values that you use.

2. Change the above code to fill the 5×5 pixel neighbourhood red around the location at which the mouse is clicked.

3. Extract a 100×100 pixel area around the location of a mouse click (called **Region of Interest – ROI**) and create and display this part of the original image in a separate new window. You may find the region of interest examples in this online tutorial helpful:

https://docs.opencv.org/master/d3/df2/tutorial_py_basic_ops.html

4. Use the function `cvtColor()` to adapt the program to first convert the colour image loaded to grayscale using the built-in OpenCV conversion. Set the code parameter of this function as **BGR2GRAY** to cope with the BGR channel ordering. Adapt the pixel access code to now report the single channel colour values.

5. Investigate the use of the `rectangle()` function (see the 'Drawing Function' section of core section of manual) to draw a 10×10 rectangle around an image point selected with a mouse click.

You may also wish to experiment with the other drawing functions available in OpenCV.

Part 3: Further Work (optional)

A good set of open source tutorials exist for the use of OpenCV with Python here:

https://docs.opencv.org/master/d6/d00/tutorial_py_root.html

The following tutorials re-affirm and extend the topics covered in this practical. They are not essential but are designed to reinforce your learning of the key concepts.

- Getting Started with Images:

https://docs.opencv.org/master/db/deb/tutorial_display_image.html

- Basic Operations on Images:

https://docs.opencv.org/master/d3/df2/tutorial_py_basic_ops.html

- Drawing Functions in OpenCV:

https://docs.opencv.org/master/dc/da5/tutorial_py_drawing_functions.html

- Mouse as a Paint-Brush:

https://docs.opencv.org/master/db/d5b/tutorial_py_mouse_handling.html